

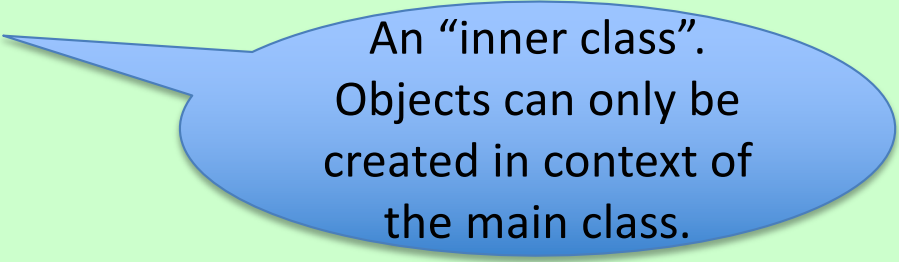
Linked Lists and Iterators

Green slides taken (w/ minor modifications) from Cynthia Lee's CS 2 slides on <http://www.peerinstruction4cs.org>, licensed under [Creative Commons Attribution-NonCommercial 4.0 International License](#).

LinkedList

```
public class MyLinkedList<T> implements List<T> {  
    private Node head;  
    //optionally: could have size
```

```
    public class Node {  
        public T value;  
        public Node next;
```



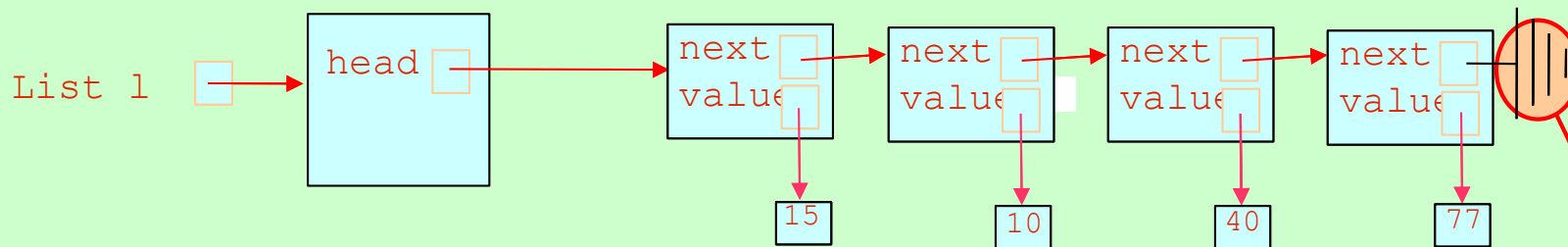
An “inner class”.
Objects can only be
created in context of
the main class.

```
        public Node(T value, Node next) {  
            this.value = value;  
            this.next = next;  
        }
```

```
    }
```

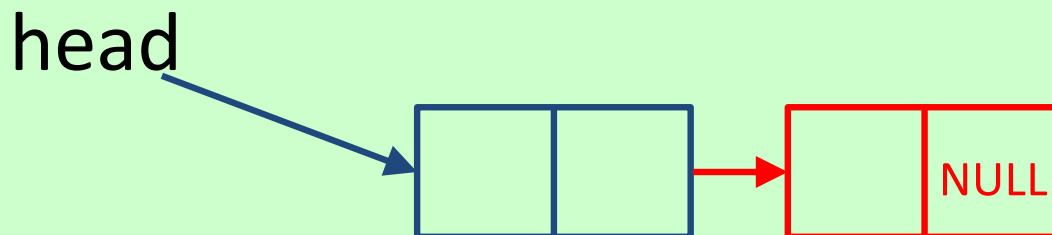
```
    ...
```

How our class looks in memory



-“head” points to first node

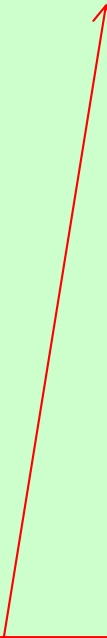
Null next pointer
means “end
of the list”



What line of code will correctly complete this method?

```
public void addFront(T newItem) {  
    Node newNode = new Node(newItem, head);  
    _____;  
}
```

- A) No line is needed. The code is correct as written.
- B) `head = head.next;`
- C) `head = newNode;`
- D) `newNode.next = head;`

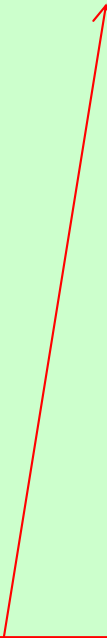


Question to think about/discuss: what is the purpose or effect of passing `head` as an argument here?

What line of code will correctly complete this method?

```
public void addFront(T newItem) {  
    Node newNode = new Node(newItem, head);  
    _____;  
}
```

- A) No line is needed. The code is correct as written.
- B) `head = head.next;`
- C) `head = newNode;`
- D) `newNode.next = head;`



Question to think about/discuss: what is the purpose or effect of passing `head` as an argument here?

We implement size() by traversing the list. Which of the following tests if curr is at the end of the list?

```
public int size() {  
    Node curr = head;  
    int count = 0;  
    while(XXX) {  
        count++;  
        ...  
    }  
    return count;  
}
```

- A. curr == head
- B. curr != null
- C. curr.next != head
- D. curr.next != null
- E. Not exactly one of the above

We implement size() by traversing the list. Which of the following tests if curr is at the end of the list?

```
public int size() {  
    Node curr = head;  
    int count = 0;  
    while(XXX) {  
        count++;  
        ...  
    }  
    return count;  
}
```

- A. curr == head
- B. curr != null
- C. curr.next != head
- D. curr.next != null
- E. Not exactly one of the above